

操作系统实验报告

姓名：李宗杭 学号：2311451 学院：软件学院

实验日期：2025.10.22 星期三 11-12 节

实验题目：目录复制

一、实验内容

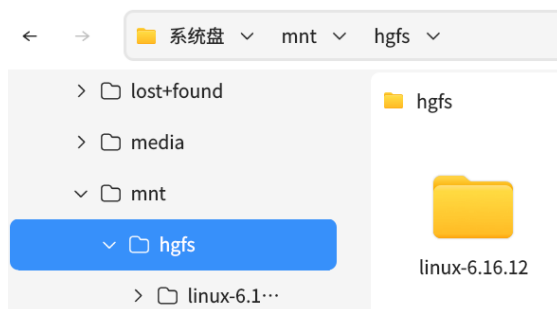
- 1、编写一个 C/C++ 程序，实现目录及其子目录的复制
- 2、使用 Linux 内核源码包 linux-6.16.12.tar.xz 解压后的目录作为测试对象。
- 3、验证复制结果是否正确

二、实现步骤

1、实验前置准备：安装开发套件

```
Asever@Asever-pc:/$ sudo apt-get install build-essential
输入密码
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
build-essential 已经是最新版 (12.8kylin1)。
升级了 0 个软件包，新安装了 0 个软件包，要卸载 0 个软件包，有 2 个软件包未被升级。
```

2、下载并解压得到 linux-6.16.12 目录



3、分别编写单进程和多进程复制程序

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <dirent.h>
6  #include <sys/stat.h>
7  #include <sys/types.h>
8  #include <fcntl.h>
9  #include <errno.h>
10
11 // 路径拼接函数：源路径 + 文件名
12 void path_join(char* dest, const char* src_path, const char* filename) {
13     strcpy(dest, src_path); // 先复制源路径到目标缓冲区
14     if (dest[strlen(dest) - 1] != '/') { // 若源路径末尾没有 '/'
15         strcat(dest, "/"); // 补充 '/'
16     }
17     strcat(dest, filename); // 拼接文件名
18 }
19
20
21 /*
22 文件拷贝函数
23 拷贝单个文件，保留源文件权限
24 参数：src_file 源文件路径，dest_file 目标文件路径
25 返回值：0 成功，-1 失败
26 */
27 int copy_file(const char* src_file, const char* dest_file) {
28     // 打开源文件（只读模式）
29     int src_fd = open(src_file, O_RDONLY);
30     if (src_fd == -1) { // 打开失败
31         perror("open src file failed");
32         return -1;
33     }
34
35     // 获取源文件状态，用于复制权限
36     struct stat src_stat;
37     lstat(src_file, &src_stat); // 获取源文件元数据（权限，大小）
38
39     // 创建目标文件，保留源文件权限
40     int dest_fd = open(dest_file, O_WRONLY | O_CREAT | O_TRUNC, src_stat.st_mode);
41     if (dest_fd == -1) { // 创建失败
42         perror("open dest file failed");
43         close(src_fd); // 关闭已打开的源文件
44         return -1;
45     }
46
47     // 循环读写数据（4KB缓冲区）
48     char buf[4096];
49     ssize_t read_len, write_len; // ssize_t支持负数（表示错误）
50     while ((read_len = read(src_fd, buf, sizeof(buf))) > 0) { // 读数据到缓冲区
51         write_len = write(dest_fd, buf, read_len); // 写缓冲区数据到目标文件
52         if (write_len != read_len) { // 写入字节数与读取不一致
53             perror("write file failed");
54             close(src_fd);
55             close(dest_fd);
56             return -1;
57         }
58     }
59
60     // 检查读操作是否出错
61     if (read_len == -1) {
62         perror("read file failed");
63         close(src_fd);
64         close(dest_fd);
65         return -1;
66     }
67
68     // 关闭文件描述符，释放资源
69     close(src_fd);
70     close(dest_fd);
71     return 0;
72 }

```

```

74  /*
75     目录拷贝函数
76     递归拷贝子目录，包括子目录，文件，符号链接
77     参数: src_file 源文件路径 , dest_file 目标文件路径
78     返回值: 0 成功 , -1 失败
79  */
80  int copy_dir(const char* src_dir, const char* dest_dir) {
81      // 获取源目录状态，用于复制权限
82      struct stat src_stat;
83      if (lstat(src_dir, &src_stat) == -1) { // 获取状态失败
84          perror("stat src dir failed");
85          return -1;
86      }
87
88      // 创建目标目录，保留源目录权限
89      // 允许目标目录已存在，忽略EEXIST错误
90      if (mkdir(dest_dir, src_stat.st_mode) == -1 && errno != EEXIST) {
91          perror("mkdir dest dir failed");
92          return -1;
93      }
94
95      // 打开源目录
96      DIR* dir = opendir(src_dir);
97      if (dir == NULL) { // 打开目录失败
98          perror("opendir src dir failed");
99          return -1;
100     }
101
102     // 遍历目录项
103     struct dirent* dir_entry; // 目录项结构体
104     char src_path[1024], dest_path[1024]; // 拼接后的完整路径
105     while ((dir_entry = readdir(dir)) != NULL) {
106         // 跳过"."和".."
107         if (strcmp(dir_entry->d_name, ".") == 0 || strcmp(dir_entry->d_name, "..") == 0) {
108             continue;
109         }
110
111         // 拼接完整路径
112         path_join(src_path, src_dir, dir_entry->d_name);
113         path_join(dest_path, dest_dir, dir_entry->d_name);
114
115         // 根据文件类型处理
116         switch (dir_entry->d_type) {
117             case DT_REG: // 普通文件
118                 if (copy_file(src_path, dest_path) == -1) {
119                     closedir(dir);
120                     return -1;
121                 }
122                 break;
123             case DT_DIR: // 目录文件
124                 if (copy_dir(src_path, dest_path) == -1) {
125                     closedir(dir);
126                     return -1;
127                 }
128                 break;
129             case DT_LNK: // 符号链接
130                 if (link(src_path, dest_path) == -1) {
131                     perror("create symlink failed");
132                     closedir(dir);
133                     return -1;
134                 }
135                 break;
136             default: // 其他文件类型暂不处理
137                 printf("unsupported file type: %s\n", src_path);
138                 break;
139         }
140     }
141
142     closedir(dir);
143     return 0;
144 }

```

```

146 int main(int argc, char* argv[]) {
147     // 检查命令行参数
148     if (argc != 3) {
149         printf("usage: %s <src_dir> <dest_dir>\n", argv[0]);    // 提示用法
150         return 1;
151     }
152
153     const char* src_dir = argv[1];    // 源目录路径
154     const char* dest_dir = argv[2];    // 目标目录路径
155
156     // 执行拷贝
157     printf("start copying %s to %s...\n", src_dir, dest_dir);
158     if (copy_dir(src_dir, dest_dir) == -1) {
159         printf("copy failed\n");
160         return 1;
161     }
162
163     printf("copy completed successfully\n");
164     return 0;
165 }

```

多进程版仅作如下主要修改

```

117     // 根据文件类型处理
118     switch (dir_entry->d_type) {
119     case DT_REG: // 普通文件
120         if (copy_file(src_path, dest_path) == -1) {
121             closedir(dir);
122             exit(1);
123         }
124         break;
125     case DT_DIR: // 目录文件
126     {
127         // 创建子进程处理子目录
128         pid_t pid = fork();
129         if (pid == -1){
130             perror("fork failed");
131             closedir(dir);
132             exit(1);
133         }
134         else if (pid == 0) {
135             // 子进程执行目录拷贝
136             copy_dir(src_path, dest_path);
137             exit(0); // 子进程成功退出
138         }
139         break;
140     }
141
142     case DT_LNK: // 符号链接
143         if (link(src_path, dest_path) == -1) {
144             perror("create symlink failed");
145             closedir(dir);
146             exit(1);
147         }
148         break;
149     default: // 其他文件类型暂不处理
150         printf("unsupported file type: %s\n", src_path);
151         break;

```

4、测试并验证



```
Asever@Asever-pc: ~/0
文件(F) 编辑(E) 视图(V) 搜索(S) 终端(T) 帮助(H)
Asever@Asever-pc:~/0$ time ./mycopy linux-6.16.12 linux-6.16.12bak
start copying linux-6.16.12 to linux-6.16.12bak...
copy completed successfully

real    3m30.067s
user    0m2.570s
sys     1m9.057s
Asever@Asever-pc:~/0$ time ./mycopy_multi linux-6.16.12 linux-6.16.12bak_multi
start copying linux-6.16.12 to linux-6.16.12bak_multi...
copy completed successfully

real    1m15.792s
user    0m6.090s
sys     1m58.838s
Asever@Asever-pc:~/0$ diff -r linux-6.16.12 linux-6.16.12bak
Asever@Asever-pc:~/0$
```

三、结论

从测试结果中可以看出，多进程的方式比单进程快了很多，并且 diff 命令无输出，表示复制后的文件夹与原文件夹无异